

University of Tehran
College of Engineering
School of Electrical and Computer Engineering

VLSI Design
First Computer Assignment
Simulation and Analysis of CMOS Basic Gates

Prepared by:
Faezeh Misaghi

Instructor:
Dr. Vahdat

2025-2026

Contents

1	Simulation Assumptions and Parameters	2
2	Basic Gate Design	2
2.1	Gate Design	2
2.1.1	INVERTER	3
2.1.2	NAND / AND	4
2.1.3	NOR / OR	5
2.1.4	XOR	6
2.1.5	MUX	6
2.2	Maximum Propagation Delay of Each Gate	7
2.3	Investigation of the Effects of Temperature and Process Corners on Maximum Propagation Delay	7
2.4	Investigation of the Effect of Transistor Dimensions on Propagation Delay .	10
3	Full Adder Design	12
3.1	Design of Full Adders	12
3.1.1	Functional Verification of Full Adders	12
3.1.2	Propagation Delay of sum and carry Bits	13
3.2	Design of 8bit Full Adder	13
3.2.1	Functional Verification of 8bit Full Adder	14
3.2.2	Propagation Delay of sum and carry Bits	15

1 Simulation Assumptions and Parameters

In the design, the following parameters and conditions were considered as the basic assumptions:

- **Fabrication Technology:** All simulations were performed based on the 90nm CMOS technology, using the library file and models corresponding to this technology for NMOS and PMOS transistors.
- **Supply Voltage:** The circuit supply voltage level (V_{DD}) was assumed to be constant and equal to 1V in all simulations.
- **Transistor Channel Length (L):** The minimum channel length for all transistors in the circuit was considered to be $L = 180\text{nm}$.
- **Input Sources:** Pulsed voltage sources were used to verify the logical operation of the gates.
- **Rise and Fall Times (T_R and T_F):** These were assumed to be 20ps by default.

2 Basic Gate Design

2.1 Gate Design

To avoid code complexity and maintain a modular design, each gate was defined as a separate subcircuit using the .SUBCKT command in a separate file named gates.sp. The initial dimensions considered for the widths of the NMOS (W_n) and PMOS (W_p) transistors in each gate are summarized in the following table:

Table 1: Initial transistor widths of the basic gates

Gate Type	W_n	W_p
INV	$1\mu\text{m}$	$2\mu\text{m}$
NAND / AND	$2\mu\text{m}$	$2\mu\text{m}$
NOR / OR	$1\mu\text{m}$	$4\mu\text{m}$
XOR	$2\mu\text{m}$	$4\mu\text{m}$
MUX	$2\mu\text{m}$	$4\mu\text{m}$

Finally, to ensure the correctness of the logical operation of the gates, we created a separate testbench file named waveform.sp and performed transient analysis (.TRAN) in it. Pulsed voltage sources with different periods were applied to the inputs so that all possible zero and one states could be tested.

The simulation results and output waveforms of each gate are presented below:

2.1.1 INVERTER

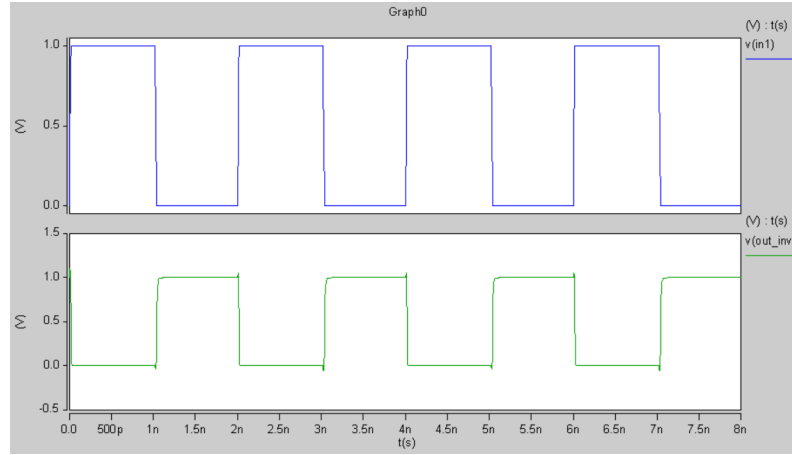


Figure 1: Input and output waveforms of the INVERTER gate

By examining the input and output waveforms, it is clearly observed that the gate output always takes the logical inverse of the input signal, confirming the correct operation of the INVERTER gate.

2.1.2 NAND / AND

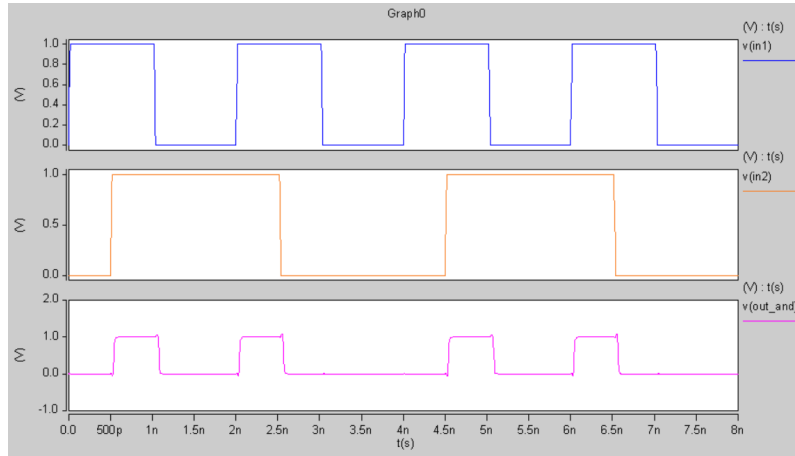


Figure 2: Input and output waveforms of the AND gate

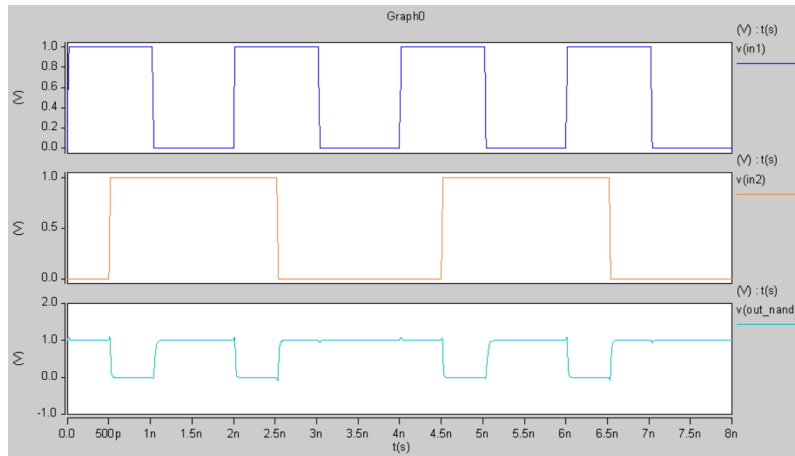


Figure 3: Input and output waveforms of the NAND gate

The waveforms show that the NAND gate output is zero only when both of its inputs are at the logical one level. The AND gate output behaves exactly as the inverse of this condition, confirming the truth tables of both gates.

2.1.3 NOR / OR

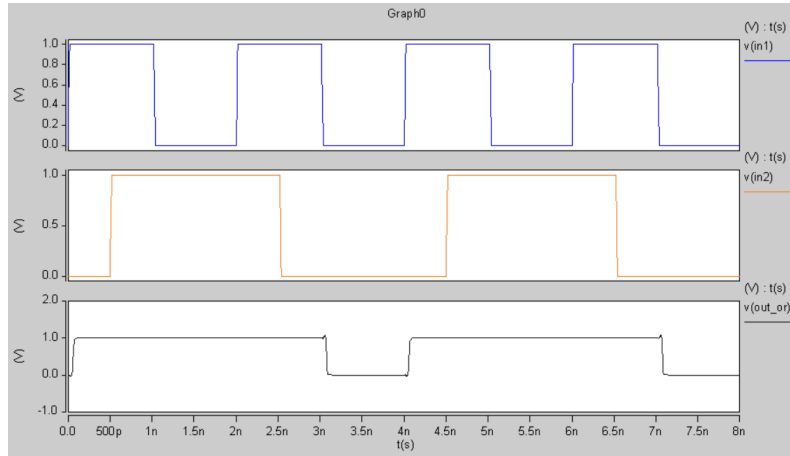


Figure 4: Input and output waveforms of the OR gate

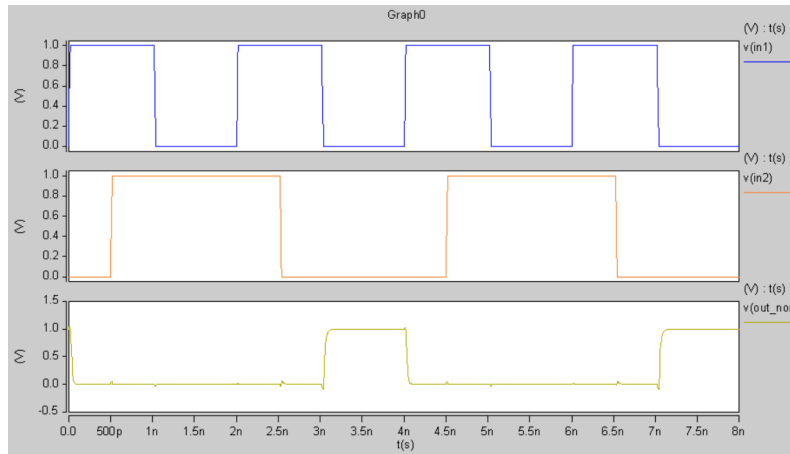


Figure 5: Input and output waveforms of the NOR gate

Based on the obtained waveforms, the OR gate output becomes logically one as soon as at least one of the inputs becomes one. On the other hand, the NOR gate output behaves exactly oppositely and is at a high voltage level only during the intervals when both inputs are zero.

2.1.4 XOR

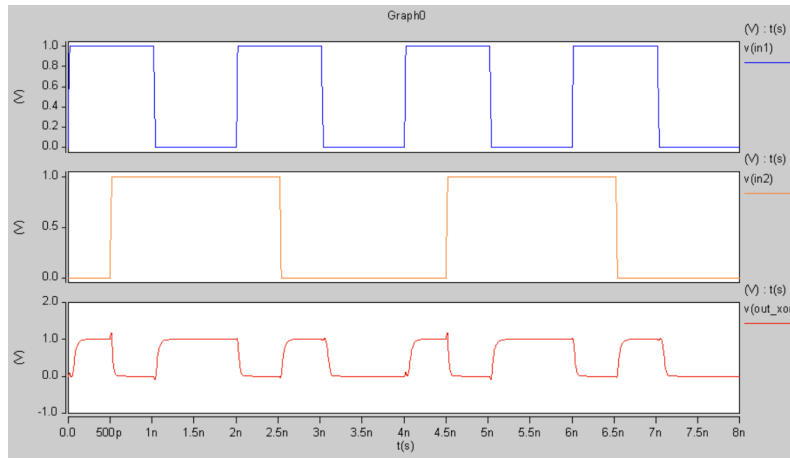


Figure 6: Input and output waveforms of the XOR gate

The simulation results clearly show that the XOR gate output becomes logically one only during the intervals when the two input signals have different logical states, one being zero and the other one, and when the inputs are identical, the output is zero.

2.1.5 MUX

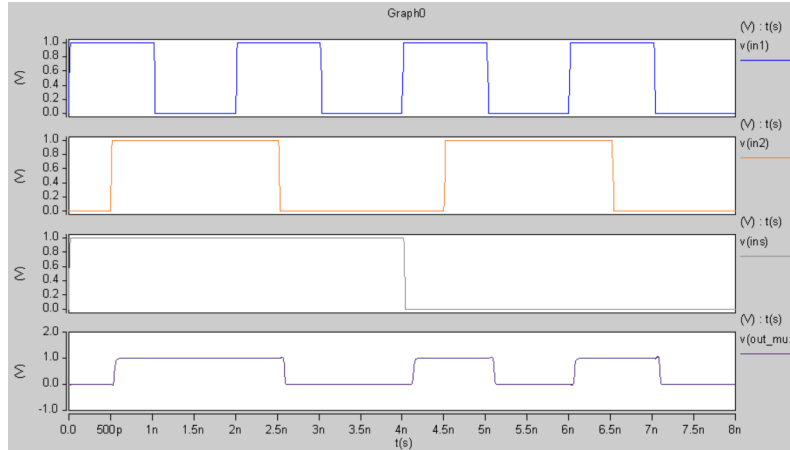


Figure 7: Waveforms of the input signals, Select line, and MUX output

According to the recorded waveforms, the multiplexer operates correctly; when the Select line is zero, the output signal exactly follows the second input, and when the select line becomes one, the output transfers the values of the first input signal to the output.

2.2 Maximum Propagation Delay of Each Gate

To find the maximum propagation delay, a path sensitivity method was used; for each gate, one of the inputs was varied with a pulse signal while the other input was kept fixed in a neutral state so that the changes would propagate to the output. This experiment was repeated for each individual input. Finally, the propagation delay for each path was calculated as the average of the rising and falling delays ($t_{pd} = \frac{t_{pLH} + t_{pHL}}{2}$), and the maximum of these values was extracted as the worst-case delay of that gate. The results are summarized in Table 2:

Table 2: Worst-case propagation delay of the basic gates

Gate Name	Worst-Case Propagation Delay
INV	10.67ps
NAND	21.75ps
NOR	25.98ps
AND	43.80ps
OR	49.55ps
XOR	70.77ps
MUX	103.70ps

2.3 Investigation of the Effects of Temperature and Process Corners on Maximum Propagation Delay

In this part of the experiment, our goal was to investigate the stability and speed of the gates under different environmental conditions and PVT variations. For this purpose, we first determined for each gate which input pattern causes the maximum delay. After identifying these patterns, we applied the same inputs to the circuit. Next, to observe the effects of process and temperature variations, we ran the simulations at 5 different temperatures (from 10°C to 70°C) and at 5 different process corners (TT, SS, FF, SF, FS). In the written code, we measured the propagation delay (t_{pd}) using the average formula, namely $t_{pd} = \frac{t_{pLH} + t_{pHL}}{2}$. Finally, we extracted the simulation data and categorized it in the tables below so that we could properly compare and analyze the effect of temperature changes and corners on the speed of each gate.

Table 3: Propagation delay of the INV gate

Corner / Temp	10°C	25°C	40°C	55°C	70°C
TT	8.61ps	8.75ps	8.89ps	9.20ps	9.44ps
SS	9.48ps	9.83ps	10.04ps	10.23ps	10.42ps
FF	7.70ps	7.86ps	8.04ps	8.20ps	8.34ps
SF	8.42ps	8.60ps	8.75ps	8.92ps	9.25ps
FS	8.83ps	9.01ps	9.20ps	9.42ps	9.58ps

Table 4: Propagation delay of the NAND gate

Corner / Temp	10°C	25°C	40°C	55°C	70°C
TT	21.04ps	21.48ps	22.20ps	22.82ps	23.37ps
SS	23.81ps	24.39ps	24.95ps	25.50ps	26.09ps
FF	18.48ps	18.97ps	19.44ps	19.93ps	20.64ps
SF	20.00ps	20.56ps	21.22ps	21.77ps	22.52ps
FS	22.25ps	22.74ps	23.28ps	23.96ps	24.60ps

Table 5: Propagation delay of the NOR gate

Corner / Temp	10°C	25°C	40°C	55°C	70°C
TT	20.79ps	21.51ps	22.27ps	22.98ps	23.64ps
SS	23.45ps	24.03ps	24.72ps	25.33ps	26.10ps
FF	18.56ps	19.10ps	19.65ps	20.21ps	21.03ps
SF	20.94ps	21.50ps	22.46ps	23.12ps	23.80ps
FS	21.05ps	21.56ps	22.06ps	22.81ps	23.64ps

Table 6: Propagation delay of the AND gate

Corner / Temp	10°C	25°C	40°C	55°C	70°C
TT	41.15ps	42.11ps	43.14ps	44.50ps	45.59ps
SS	47.41ps	48.59ps	49.70ps	50.79ps	52.02ps
FF	35.44ps	36.38ps	37.31ps	38.35ps	39.29ps
SF	40.66ps	41.80ps	42.98ps	44.17ps	45.46ps
FS	41.80ps	42.70ps	43.77ps	44.73ps	45.93ps

Table 7: Propagation delay of the OR gate

Corner / Temp	10°C	25°C	40°C	55°C	70°C
TT	42.49ps	43.75ps	45.08ps	46.39ps	47.55ps
SS	48.49ps	49.70ps	51.10ps	52.53ps	53.96ps
FF	37.18ps	38.32ps	39.43ps	40.60ps	41.88ps
SF	42.86ps	44.05ps	45.50ps	46.71ps	47.98ps
FS	42.52ps	43.77ps	44.81ps	46.13ps	47.60ps

Table 8: Propagation delay of the XOR gate

Corner / Temp	10°C	25°C	40°C	55°C	70°C
TT	64.82ps	66.36ps	68.05ps	69.72ps	71.44ps
SS	74.34ps	76.43ps	78.20ps	79.95ps	81.99ps
FF	56.04ps	57.55ps	59.03ps	60.57ps	62.22ps
SF	63.80ps	65.52ps	67.38ps	69.10ps	70.92ps
FS	66.61ps	68.05ps	69.76ps	71.56ps	73.18ps

Table 9: Propagation delay of the MUX gate

Corner / Temp	10°C	25°C	40°C	55°C	70°C
TT	96.31ps	98.82ps	101.30ps	104.10ps	106.90ps
SS	111.80ps	114.70ps	117.50ps	120.50ps	123.40ps
FF	82.82ps	85.35ps	87.76ps	90.28ps	92.82ps
SF	96.57ps	99.17ps	102.00ps	104.70ps	107.40ps
FS	96.68ps	99.16ps	101.90ps	104.00ps	106.90ps

Looking at all the tables, a clear trend can be observed: as the temperature increases from 10°C to 70°C, the propagation delay (t_{pd}) increases for all gates and all corners.

Process variations have a strong effect on transistor speed:

- **Corner SS (Slow-Slow):** For all gates, the maximum delay corresponds to this condition. At this corner, both NMOS and PMOS transistors are in their slowest state.
- **Corner FF (Fast-Fast):** The minimum delay in all tables corresponds to this corner. Both NMOS and PMOS are in their fastest state.
- **Corner TT (Typical-Typical):** This is the nominal and average condition, with its delay always lying between SS and FF.
- **Corners FS and SF:** These corners (where one side is fast and the other is slow) have delays close to the TT condition, but due to the asymmetry in Pull-up and Pull-down strength, they can cause a large difference between t_{pHL} and t_{pLH} .

2.4 Investigation of the Effect of Transistor Dimensions on Propagation Delay

In this part of the project, the goal is to investigate the effect of the dimensional ratio of PMOS to NMOS transistors on the propagation delay of the basic gates INV, NAND, and NOR. For this purpose, we kept the width of the transistors in the Pull-down network fixed at $W_n = 1\mu\text{m}$ and defined the width of the transistors in the Pull-up network as a multiple of it ($W_p = \beta \times W_n$).

Using the command `.TRAN ... SWEEP beta 1 5 1`, we varied β from 1 to 5 and extracted the values of t_{pLH} , t_{pHL} , and t_{pd} . The results for the propagation delay (t_{pd}) are presented in the following table:

Table 10: Effect of the dimensional ratio (β) on the propagation delay of different gates

β	INV t_{pd}	NAND t_{pd}	NOR t_{pd}
1	18.12 ps	34.60 ps	35.49 ps
2	17.45 ps	31.13 ps	34.40 ps
3	18.14 ps	31.66 ps	36.18 ps
4	19.06 ps	33.02 ps	38.50 ps
5	20.10 ps	34.83 ps	40.86 ps

By examining the raw t_{pHL} and t_{pLH} data, three important trends can be observed in the circuit behavior:

- t_{pLH} : As β increases from 1 to 5, t_{pLH} decreases in all gates.
- t_{pHL} : In contrast, as β increases, the t_{pHL} delay continuously increases.
- t_{pd} : As shown in the table above, the average delay does not follow a linear trend as β increases. It initially decreases, reaches its minimum value at $\beta = 2$, and then starts increasing again.

What is the reason for these changes?

- **Reason for the decrease in t_{pLH} :** The PMOS transistors are responsible for charging the output capacitor. As β increases, the PMOS channel width becomes larger, and consequently its equivalent resistance (R_p) decreases. Lower resistance causes more current to be injected into the output, allowing the signal to rise faster.
- **Reason for the increase in t_{pHL} :** Increasing the width of the PMOS transistors increases the area of their Drain region. This increases the Diffusion Capacitances at the output node. Since the NMOS transistor width is kept constant ($W_n = 1\mu\text{m}$) in this experiment, this transistor now has to discharge a larger capacitance, which naturally takes more time.
- **Reason for the minimum delay at $\beta = 2$:** In semiconductor physics, the mobility of electrons in silicon is approximately 2 to 3 times that of holes. At $\beta = 1$, the

PMOS transistor is weaker than the NMOS transistor ($t_{pLH} > t_{pHL}$). By choosing $\beta = 2$, the drive strengths of PMOS and NMOS become approximately symmetric, providing the best operating point for minimizing the overall delay. If β becomes too large, a phenomenon called Self-Loading occurs; that is, the detrimental effect of increased parasitic capacitance dominates the positive effect of reduced resistance, and the circuit eventually becomes slower.

3 Full Adder Design

3.1 Design of Full Adders

In this part of the project, the goal was to implement and simulate two different structures for a one-bit full adder:

- **Structure A:** Using XOR and NAND gates
- **Structure B:** In the form of All-NAND

A dedicated library file named `adders.sp` was created. In this file, two SUBCKTs named `FullAdder_A` and `FullAdder_B` were defined, each implementing its corresponding structure. The basic gates designed in the previous stage were required to build these adders; therefore, the `gates.sp` file was called within this library so that these subcircuits could access the basic gates.

3.1.1 Functional Verification of Full Adders

To test and verify the operation of these two structures, we wrote a testbench file. We had to evaluate the circuit for all states of the truth table of a Full Adder. Therefore, we generated the inputs A , B , and C_{in} using pulsed voltage sources and set their periods differently so that all combinations of logical zeros and ones would be generated. Finally, we defined a transient analysis using the `.TRAN` command to examine the changes in the inputs and outputs over a specified time interval. The waveforms and results obtained from this simulation are presented below.

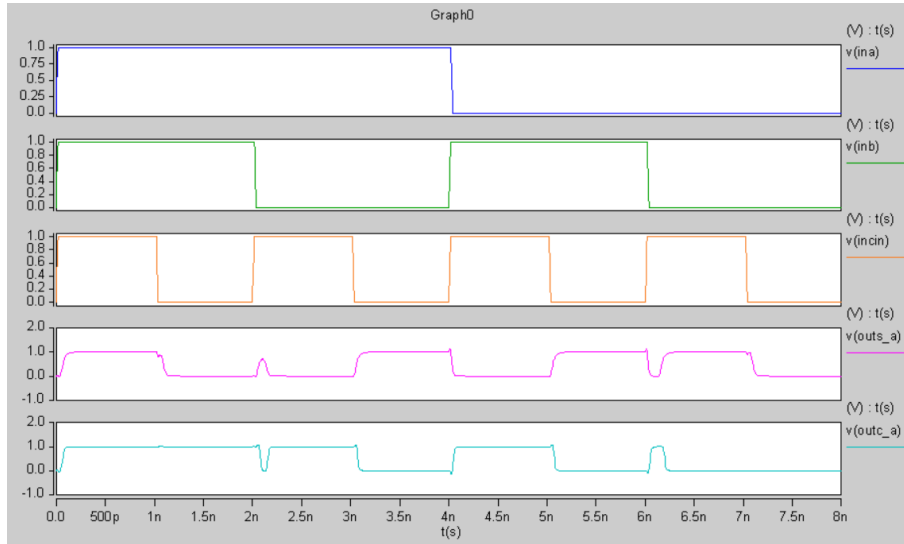


Figure 8: Waveform of Structure A

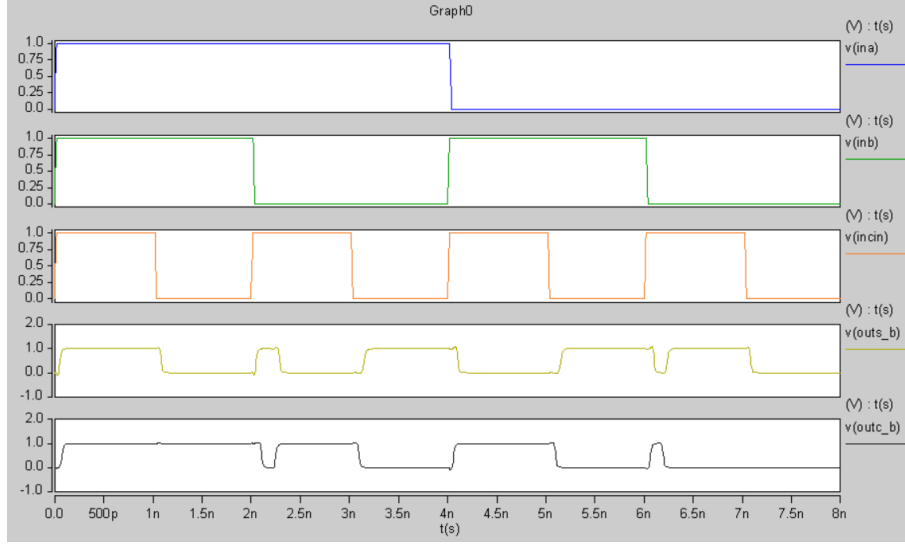


Figure 9: Waveform of Structure B

3.1.2 Propagation Delay of sum and carry Bits

In this part of the experiment, our goal was to find the maximum propagation delay (Worst-Case Delay) for the two full-adder structures. To accurately measure the delay of each path in the INSTANTIATIONS section, we called each fulladder structure three times. In each call, the variable pulse signal was applied to only one of the inputs A , B , or C_{in} , while the other inputs were connected to fixed voltages. Using the .measure command for each input path and for both outputs Sum and Cout, the values of t_{pLH} and t_{pHL} were calculated.

Table 11: Maximum propagation delay for the full-adder structures

Delay	Full Adder A	Full Adder B
t_{pd_S}	153.4 ps	206.2 ps
t_{pd_Cout}	175.0 ps	168.5 ps
t_{pd_FA}	175.0 ps	206.2 ps

3.2 Design of 8bit Full Adder

To implement the 8-bit Ripple Carry Adder, we had to choose one of the two structures, A and B, as the basic block. At first glance, and considering the results of the delay measurement section, Structure A might seem to be the better choice because its worst-case delay was considerably lower than that of Structure B. However, a very important concept in the architecture of an RCA is the Critical Path. In this adder, each Full Adder must wait to receive C_{in} from the previous Full Adder to calculate its final output. Therefore, the main speed bottleneck in this circuit is not the delay in generating output S , but the delay in generating output C_{out} . By revisiting the delay simulation results,

we find that Structure B performs better in generating the C_{out} output. Therefore, to have a faster and more optimized 8-bit adder, we selected Structure B so that the critical path would be traversed as quickly as possible and the circuit would reach a stable state in the shortest possible time. We wrote the code for the 8-bit adder in a separate file named `adder_8bit.sp`. In this file, eight `FullAdder_B` blocks are connected in cascade to construct the adder.

3.2.1 Functional Verification of 8bit Full Adder

To ensure the correct operation of the designed 8-bit adder circuit, a separate testbench was implemented in the file `adder8b_waveform.sp`. The purpose of this testbench is to examine the circuit outputs (S_0 through S_7 and C_{out}) for different inputs and simultaneously observe the input and output waveforms. As requested in the project instructions, to better display and simplify the analysis of the outputs, the input carry bit was kept at zero throughout the simulation ($C_{in} = 0$). For a comprehensive examination of the circuit, three different input scenarios were applied to the circuit using PWL voltage sources, as described below:

- **Case 1 ($A = 255$ and $B = 0$):** In this case, all bits of A are one and all bits of B are zero. The circuit is expected to show the same value, 255, at output S without generating any carry, and C_{out} should be zero.
- **Case 2 ($A = 255$ and $B = 1$):** In this scenario, A is equal to 255 and B is equal to 1. This is the most important case for testing a cascaded adder because adding the first two bits generates a carry, and this carry must propagate through all 8 stages (the Ripple Effect) so that the final output S becomes 0 and C_{out} becomes 1.
- **Case 3 ($A = 15$ and $B = 15$):** In this case, the four least significant bits of both inputs are one (00001111) and their four most significant bits are zero. The expected sum is 30 (00011110), and C_{out} should remain zero.

Based on the simulation plots, the correct operation of the circuit in the three specified time intervals is verified as follows:

- **Interval 0 to 2 ns (case $A = 255$ and $B = 0$):** As can be seen, all sum signals (S_0 through S_7) are at the 1 V voltage level (logic 1), and the C_{out} signal is zero. This output correctly corresponds to the number 255.
- **Interval 2 to 4 ns (case $A = 255$ and $B = 1$):** At 2 ns, when B_0 changes from 0 to 1 V, the carry propagation phenomenon (Ripple Effect) clearly occurs. Zooming in around this time shows that the signals S_0 through S_7 do not become zero simultaneously; instead, they go to 0 V sequentially and with delay, and finally C_{out} becomes one (equivalent to the number 256). The time interval between the rising edge of B_0 and the rising edge of C_{out} represents the worst-case delay of the circuit.
- **Interval 4 to 6 ns (case $A = 15$ and $B = 15$):** After applying the input changes at 4 ns, the signals S_1 through S_4 are at the 1 V level, while the other sum bits and C_{out}

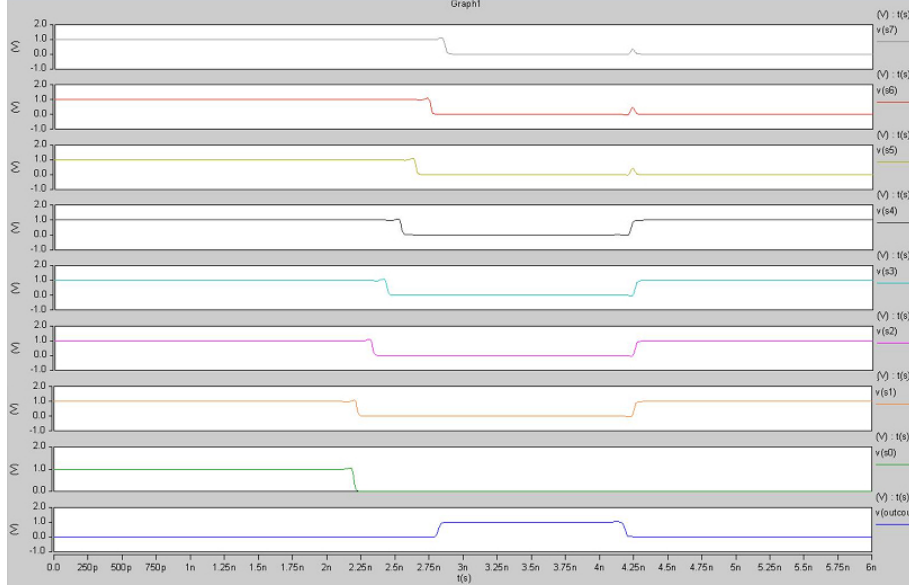


Figure 10: Waveform of the 8bit full adder

become zero. This output pattern (00011110) is exactly equivalent to the expected sum, namely the number 30.

The transient simulation results show that in all three cases, after the gate delay has elapsed, the circuit correctly reaches the expected logic levels, and the correct operation of the 8-bit adder is fully verified.

3.2.2 Propagation Delay of sum and carry Bits

To measure the maximum delay time in the circuit, we implemented the testbench `adder8b_delay.sp`. In a Ripple Carry Adder, the longest delay path occurs when a Carry generated at the first bit is forced to pass through all stages and propagate to the last bit. To create this worst-case condition in the testbench, we set input A to all ones (11111111) and then added one to the first bit of the other input. This causes the carry to propagate in a domino-like manner through all 8 stages and ultimately affect the final S_7 and C_{out} .

Table 12: Maximum sum and carry delay

Measured Parameter	Software Output (seconds)	Equivalent Value (picoseconds)
Worst-case sum delay	8.506×10^{-10}	850.6 ps
Worst-case carry delay	8.094×10^{-10}	809.4 ps

As shown in the table, the total circuit delay increases significantly in the 8-bit configuration. In the previous simulations, we saw that a one-bit Full Adder cell (Structure B) had a delay of 206.2 ps for the Sum output and 168.5 ps for the C_{out} output. The reason for this difference is that in the 8-bit circuit, the carry delay of the stages accumulates. The S_7 output must wait for the carry to pass through the previous 7 stages before it can

be calculated, and therefore the maximum total circuit delay (850.6 ps) corresponds to it. These results exactly confirm the reason for selecting Structure B for the 8-bit design; in RCA circuits, the critical path is determined by the propagation of C_{out} . Since Structure B had a lower C_{out} delay than Structure A, using it minimized the effect of delay accumulation across these 8 stages and kept the overall circuit speed optimized.